

My report Title

*Phase II project report submitted in partial fulfilment of the requirements for
the award of the degree of*

Bachelor of Technology

in

Computer Science & Design

Submitted by

My Name

Teammate1 Name

Teammate2 Name

Teammate3 Name



Focus on Excellence

Federal Institute of Science And Technology (FISAT)®
Angamaly, Ernakulam

Affiliated to

APJ Abdul Kalam Technological University
CET Campus, Thiruvananthapuram
April 2026

FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY
(FISAT)

Mookkannoor(P.O), Angamaly-683577



CERTIFICATE

This is to certify that the project phase II report for the project entitled “**My report Title**” is a bonafide report of the project presented during VIIIth semester (CXD416 - Project Phase II) by **My Name(FITCSD2001)**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science & Design during the academic year 2025-26.

Dr. Paul P Mathai

Project Coordinator

Project Guide

Head of the Department

ABSTRACT

Abstract is a condensed version of your whole project.

- It should cover the major parts of the project.
- It should not contain any excess wordiness or unnecessary information.
- It should be readable and well organized.
- It flows smoothly between the parts.

An abstract convey the following information:

- The purpose of the project, identifying the area of study to which it belongs.
- The problem addressed.
- The research problem that motivates the project.
- Its importance/novelty
- The methods/approach used to address this problem, documents or evidence analyzed, highlighting novelty, if any.
- the major results obtained
- The conclusions reached or, if the work is in progress, what the preliminary results of the investigation suggest.
- The significance of the project. Why are the results useful? What is new to our understanding as the result of your inquiry?

Contribution by Author

Author1 Contribution

Author2 Contribution

Author3 Contribution

Author4 Contribution Goes Here

Student1 Name

Student2 Name

Student3 Name

Student4 Name

ACKNOWLEDGMENT

Your Acknowledgment Goes Here

In the acknowledgment section, you will recognize the support and guidance of the people who have helped you with any aspects of the project.

Student 1

Contents

List of Figures

List of Tables

Chapter 1

Literature Review

1.1 Related Work

Serverless computing, also known as Function-as-a-Service (FaaS), has been extensively studied in recent years. According to research by Baldini et al. (2017), serverless computing represents a natural evolution of cloud computing where developers focus solely on code while the platform handles infrastructure.

1.1.1 Serverless Platforms

AWS Lambda (2014) pioneered the serverless space, followed by Azure Functions and Google Cloud Functions. These platforms provide event-driven execution with automatic scaling and pay-per-use pricing.

1.1.2 Orchestration Solutions

Several solutions address serverless orchestration:

- AWS Step Functions: Amazon’s proprietary workflow orchestration service
- Azure Durable Functions: Microsoft’s stateful workflow implementation
- Google Workflows: Google’s orchestration solution
- Apache OpenWhisk: Open-source serverless platform with orchestration
- Knative: Kubernetes-based serverless platform

1.1.3 Academic Research

Research by Fox et al. (2020) analyzed serverless computing patterns and challenges. They identified orchestration as a key area requiring standardization. Vahid et al. (2019) proposed workflow patterns specifically designed for serverless architectures.

1.2 Comparison of Related Works

Platform	Workflow Definition	State Management	Error Handling	Vendor Lock-in
AWS Step Functions	JSON/Amazon States Language	Built-in	Retry/Catch	High
Azure Durable Functions	C#/JavaScript	Built-in	Retry	High
Apache OpenWhisk	YAML/Composite	External	Custom	Low
Our Service	YAML	Built-in	Customizable	Low

Table 1.1: Comparison of Serverless Orchestration Solutions

1.3 Proposed System

Based on the analysis of existing solutions, our proposed system aims to provide:

- Vendor-agnostic workflow definitions
- Declarative workflow specification using YAML
- Built-in state persistence
- Comprehensive error handling with retry policies
- Real-time execution monitoring
- Integration with existing serverless functions